CONFIDENTIAL

Vulnerability Assessment Report

Google Gruyere Web Application Security Audit

Target: google-gruyere.appspot.com

Assessment Date: March 22, 2026

Prepared By: Purnendu Rai
CyberSecurity Intern – Future Interns

Classification: Read-Only Security Assessment

This report contains sensitive security information.

Table of Contents

1. Executive Summary

2. Assessment Overview

2.1 Scope and Objectives

2.2 Methodology

2.3 Target Information

3. Risk Summary

4. Detailed Findings

4.1 Critical Severity

4.2 High Severity

4.3 Medium Severity

4.4 Low Severity

5. Recommendations

6. Conclusion

7. Appendix

1. Executive Summary

Key Findings at a Glance

This vulnerability assessment was conducted on the Google Gruyere web application, an intentionally vulnerable application designed for security training purposes. The assessment identified **10 security vulnerabilities** across various categories, including one **CRITICAL** severity issue that requires immediate attention.

Severity	Count	Risk Level
Critical	1	Immediate action required
High	3	Address within 30 days
Medium	5	Address within 90 days
Low	1	Address as resources permit

The most critical finding is **VULN-001: Sensitive Information Exposure in URL Parameters**, where user passwords are transmitted in plaintext as URL parameters during account creation. This exposes credentials in browser history, server logs, proxy logs, and referrer headers.

The assessment followed a read-only methodology using passive scanning techniques, including security header analysis, configuration review, and examination of application behavior without exploiting vulnerabilities.

Primary Recommendations

1. **Immediate:** Stop transmitting passwords in URL parameters; use POST requests with HTTPS
2. **High Priority:** Implement security headers (CSP, X-Frame-Options, HSTS)

3. **High Priority:** Implement input validation and output encoding to prevent XSS
4. **Medium Priority:** Add CSRF tokens to all state-changing operations
5. **Medium Priority:** Implement rate limiting on authentication endpoints

2. Assessment Overview

2.1 Scope and Objectives

This vulnerability assessment was conducted with the following scope and objectives:

Assessment Type	Read-Only Security Assessment (Passive Scanning)
Target Application	Google Gruyere Web Application
Target URL	https://google-gruyere.appspot.com/5237362078504217857854172520119198324 59
Assessment Date	March 22, 2026
Assessor	Future Interns - Cybersecurity Task 1

Scope Limitations

- Public-facing pages only
- Passive scanning and observation only
- No exploitation or active attacks performed
- No brute force or denial-of-service testing
- No access to backend systems or databases

2.2 Methodology

The assessment followed industry-standard methodologies including the OWASP Testing Guide and security best practices. The following techniques were employed:

> **Security Header Analysis**
>
> Examined HTTP response headers to identify missing security controls such as Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security.

> **Configuration Review**
>
> Analyzed application configuration, including authentication mechanisms, session management, and access controls.

> **Client-Side Analysis**
>
> Reviewed HTML source code, JavaScript files, and client-side functionality for security issues.

> **Behavioral Analysis**
>
> Observed application behavior during normal operations to identify information disclosure and logic flaws.

2.3 Target Information

Google Gruyere is an intentionally vulnerable web application developed by Google for security training and education purposes. The application contains various security vulnerabilities designed to help developers and security professionals learn about web application security.

3. Risk Summary

The following table summarizes all identified vulnerabilities by severity and category:

ID	Vulnerability	Severity	OWASP Category
VULN-001	Sensitive Information Exposure in URL Parameters	**Critical**	A05: Security Misconfiguration
VULN-002	Missing Security Headers	**High**	A05: Security Misconfiguration
VULN-003	Cross-Site Scripting (XSS) Vulnerabilities	**High**	A03: Injection

VULN-004	Insecure Password Storage and Transmission	**High**	A02: Cryptographic Failures
VULN-005	Cross-Site Request Forgery (CSRF)	**Medium**	A01: Broken Access Control
VULN-006	Path Traversal Vulnerability	**Medium**	A01: Broken Access Control
VULN-007	Insecure File Upload Functionality	**Medium**	A05: Security Misconfiguration
VULN-008	Information Disclosure via Error Messages	**Low**	A05: Security Misconfiguration
VULN-009	Missing Rate Limiting on Authentication	**Medium**	A07: Authentication Failures
VULN-010	Client-Side State Manipulation	**Medium**	A01: Broken Access Control

4. Detailed Findings

4.1 Critical Severity

VULN-001 Sensitive Information Exposure in URL Parameters

CRITICAL

CVSS Score: 7.5 **Category:** Information Disclosure **OWASP:** A05:2021 - Security Misconfiguration

Description

The application transmits sensitive user credentials (passwords) as URL parameters during account creation. The password is visible in plaintext in the URL, which exposes it to multiple attack vectors.

Evidence

```
URL observed during account creation:
/saveprofile?action=new&uid=testuser123&pw=TestPass123&is_author=True
```

Impact

Passwords are exposed in browser history, server logs, proxy logs, and referrer headers. Anyone with access to these locations (including browser users, system administrators, or attackers who compromise logs) can obtain user credentials in plaintext.

Remediation

Use POST requests with form data in the request body instead of URL parameters. Never transmit passwords in URLs. Ensure HTTPS is enforced for all authentication operations.

Business Impact

High - Direct credential exposure leading to account takeover and potential data breach.

4.2 High Severity

VULN-002 Missing Security Headers

HIGH

CVSS Score: 6.5 **Category:** Security Misconfiguration **OWASP:** A05:2021 - Security Misconfiguration

Description

The application lacks critical security headers that protect against common web attacks. Missing headers include Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, and X-XSS-Protection.

Impact

Without these headers, the application is vulnerable to XSS attacks, clickjacking, MIME sniffing attacks, and man-in-the-middle attacks. Attackers can exploit these weaknesses to steal session cookies, perform unauthorized actions, or inject malicious content.

Remediation

Implement the following security headers:
- Content-Security-Policy: default-src 'self'
- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff
- Strict-Transport-Security: max-age=31536000; includeSubDomains
- X-XSS-Protection: 1; mode=block

VULN-003 Cross-Site Scripting (XSS) Vulnerabilities

HIGH

CVSS Score: 6.1 **Category:** Injection **OWASP:** A03:2021 - Injection

Description

The application allows HTML in user snippets and lacks proper output encoding, making it vulnerable to stored and reflected XSS attacks. User input is not properly sanitized before being displayed to other users.

Evidence

Application documentation confirms: "HTML in Snippets: Users can include a limited subset of HTML in their snippets."

Impact

Attackers can inject malicious scripts that execute in victims' browsers, enabling session hijacking, credential theft, defacement, and redirection to malicious sites.

Remediation

Implement strict input validation and output encoding. Use a whitelist approach for allowed HTML tags. Implement Content Security Policy to restrict script execution. Sanitize all user input using libraries like DOMPurify before displaying it.

VULN-004 Insecure Password Storage and Transmission

HIGH

CVSS Score: 7.0 **Category:** Cryptographic Failure **OWASP:** A02:2021 - Cryptographic Failures

Description

Passwords are transmitted in plaintext and displayed in regular text fields instead of password input types. This makes passwords visible to anyone looking at the screen.

Evidence

Login and signup forms use regular text inputs for passwords instead of type="password". Passwords are visible in URL parameters during account creation.

Impact

Passwords can be easily observed by shoulder surfing, captured in browser history, or intercepted if HTTPS is not enforced. This significantly increases the risk of credential compromise.

Remediation

Use password input types (type="password") to mask passwords during entry. Implement HTTPS everywhere. Use strong password hashing algorithms (bcrypt, Argon2) with salting on the server side. Never store passwords in plaintext.

4.3 Medium Severity

VULN-005 Cross-Site Request Forgery (CSRF) Vulnerability

MEDIUM

CVSS Score: 5.4 **Category:** Session Management **OWASP:** A01:2021 - Broken Access Control

Description

The application does not implement CSRF tokens for state-changing operations, making it vulnerable to CSRF attacks where attackers can trick authenticated users into performing unwanted actions.

Impact

Attackers can trick authenticated users into performing actions like changing passwords, posting content, or modifying account settings without their knowledge.

Remediation

Implement CSRF tokens for all state-changing operations (POST, PUT, DELETE). Use SameSite cookie attributes (Strict or Lax). Validate the Origin and Referer headers for sensitive operations.

VULN-006 Path Traversal Vulnerability

MEDIUM

CVSS Score: 5.3 **Category:** Directory Traversal **OWASP:** A01:2021 - Broken Access Control

Description

The application is vulnerable to path traversal attacks that could allow attackers to access files outside the intended directory by using "../" sequences in file paths.

Impact

Attackers may access sensitive files, configuration files, source code, or system files outside the web root directory, leading to information disclosure or system compromise.

Remediation

Validate and sanitize all file paths. Use allowlists for accessible files. Remove or encode path traversal sequences. Implement proper access controls and chroot jails where appropriate.

VULN-007 Insecure File Upload Functionality

MEDIUM

CVSS Score: 5.0 **Category:** File Upload **OWASP:** A05:2021 - Security Misconfiguration

Description

File upload functionality at /upload.gtl lacks visible file type validation, size restrictions, or malware scanning, potentially allowing malicious file uploads.

Impact

Attackers could upload malicious files, web shells, or executable code that could lead to server compromise, remote code execution, or malware distribution.

Remediation

Validate file types using magic numbers (not just extensions). Restrict allowed file extensions. Implement file size limits. Scan uploads with antivirus. Store files outside web root with proper access controls. Rename uploaded files to prevent execution.

VULN-009 Missing Rate Limiting on Authentication

MEDIUM

CVSS Score: 5.3 **Category:** Brute Force **OWASP:** A07:2021 - Authentication Failures

Description

No rate limiting or account lockout mechanism is observed on the login functionality at /login, allowing unlimited authentication attempts.

Impact

Attackers can perform brute force or dictionary attacks to guess user credentials. Automated tools can attempt thousands of password combinations without restriction.

Remediation

Implement rate limiting (e.g., max 5 attempts per minute). Enable account lockout after failed attempts. Implement CAPTCHA after failed login attempts. Consider multi-factor authentication for sensitive accounts.

VULN-010 Client-Side State Manipulation

MEDIUM

CVSS Score: 5.4 **Category:** Access Control **OWASP:** A01:2021 - Broken Access Control

Description

The application relies on client-side data that can be manipulated by users to escalate privileges or access unauthorized data. The "is_author" parameter in the URL is an example of client-controlled state.

Impact

Users may be able to elevate their privileges (e.g., from regular user to author or admin) or access data belonging to other users by manipulating client-side parameters.

Remediation

Implement server-side validation for all access control decisions. Never trust client-side data for authorization. Store user roles and permissions server-side in the session or database.

4.4 Low Severity

VULN-008 Information Disclosure via Verbose Error Messages

LOW

CVSS Score: 3.7 **Category:** Information Disclosure **OWASP:** A05:2021 - Security Misconfiguration

Description

The application exposes source code through the /code/ endpoint, allowing attackers to understand application logic and find additional vulnerabilities.

Impact

Attackers can analyze source code to find additional vulnerabilities, understand application logic, and craft more effective attacks.

Remediation

Disable debug mode in production. Implement custom error pages that don't reveal system information. Restrict access to source code, configuration files, and debugging endpoints.

5. Recommendations

Immediate Actions (Critical - Within 24 Hours)

1. **Fix Password Transmission:** Immediately change the account creation form to use POST method with passwords in the request body, not URL parameters.
2. **Enable HTTPS:** Ensure all authentication and sensitive operations use HTTPS with proper TLS configuration.

Short-Term Actions (High Priority - Within 30 Days)

1. **Implement Security Headers:** Add Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security headers.
2. **Fix XSS Vulnerabilities:** Implement proper input validation and output encoding. Use a whitelist approach for allowed HTML.
3. **Secure Password Fields:** Change all password inputs to type="password" to mask user input.
4. **Implement CSRF Protection:** Add CSRF tokens to all state-changing forms and validate them on the server.

Medium-Term Actions (Medium Priority - Within 90 Days)

1. **Implement Rate Limiting:** Add rate limiting on authentication endpoints to prevent brute force attacks.
2. **Secure File Uploads:** Implement file type validation, size limits, and malware scanning for uploaded files.
3. **Fix Path Traversal:** Validate and sanitize all file paths to prevent directory traversal attacks.
4. **Server-Side Authorization:** Move all authorization checks to the server side. Never trust client-provided parameters for access control.

Long-Term Improvements

1. **Security Audit:** Conduct a comprehensive security audit with active testing (with proper authorization).
2. **Security Training:** Provide security training for developers on secure coding practices.

3. **Regular Assessments:** Implement a schedule for regular security assessments and penetration testing.
4. **Bug Bounty Program:** Consider implementing a responsible disclosure program for security researchers.

6. Conclusion

This vulnerability assessment of the Google Gruyere web application identified 10 security vulnerabilities ranging from Critical to Low severity. The most critical issue is the exposure of passwords in URL parameters, which requires immediate remediation.

The findings align with the application's stated purpose as an intentionally vulnerable training application. However, if similar patterns exist in production applications, they would pose significant security risks.

The assessment demonstrates the importance of:

- Never transmitting sensitive data in URL parameters
- Implementing proper security headers
- Validating and sanitizing all user input
- Implementing server-side authorization checks
- Using secure authentication mechanisms

Organizations should use these findings as a learning opportunity to identify and remediate similar vulnerabilities in their own applications. Regular security assessments, secure coding training, and defense-in-depth strategies are essential for maintaining application security.

7. Appendix

A. Assessment Methodology

This assessment followed the OWASP Web Security Testing Guide (WSTG) and used the following techniques:

- Passive security scanning and observation
- Security header analysis
- Configuration review

- Client-side code analysis
- Behavioral analysis of application functionality

B. Tools Used

- Browser Developer Tools (Chrome DevTools)
- Manual code review
- HTTP header inspection
- Python for documentation and analysis

C. References

- OWASP Top 10:2021 - https://owasp.org/Top10/
- OWASP Web Security Testing Guide - https://owasp.org/www-project-web-security -testing-guide/
- Google Gruyere Documentation - https://google-gruyere.appspot.com/
- CVSS v3.1 Calculator - https://www.first.org/cvss/calculator/3.1

D. Disclaimer

This assessment was conducted in a read-only manner using passive scanning techniques only. No active exploitation, brute force attacks, or denial-of-service testing was performed. The assessment was conducted on an intentionally vulnerable application designed for educational purposes.

Important: The vulnerabilities identified in this report are specific to the Google Gruyere training application. Similar vulnerabilities in production systems could result in significant security breaches and should be addressed immediately.